

# ITERATIVE MERGE SORT (BOTTOM-UP / NO RECURSION)

Exact Line-by-Line Code Structure with In-Depth Explanatory Comments

```
#include <stdio.h> // Include standard I/O library for console input and output
int main() // Execution entry point
{

    int n; // Variable to store total number of elements in the array
    printf("enter the number of elements in the array\n"); // Prompt user to enter array size
    scanf("%d",&n); // Read total element count from standard input
    int a[n]; // Primary array storing original and intermediate sorted data
    int b[n]; // Auxiliary temporary array for merged subarrays
    printf("enter\n"); // Prompt user to input array values
    for (int i=0;i<n;i++) // Loop over all indices from 0 up to n - 1
    {
        scanf("%d",&a[i]); // Read each integer into array a
    }

    int s = 1; // Initial sub-array window size (1, 2, 4, 8...) for bottom-up
merge
    while (s < n) // Continue merging passes until sub-array size covers full
array
    {
        int z = 0; // Tracker index for inserting merged items into buffer array b
        for (int i = 0; i < n; i = i + (2 * s)) // Step across array in chunks of pairs of size 2*s
        {
            int j = i; // Start index of the first (left) subarray
            int k = i + s; // Start index of the second (right) subarray
            if(k>n) // Guard condition to ensure right start does not exceed array
bounds
            {
                k=n; // Clamp right subarray start to array length n
            }
            int mid = k; // Mark ending boundary of left subarray / start of right
subarray
            int l = k + s; // Calculate upper bound of right subarray
            if(l>n) // Guard condition to prevent exceeding boundary of array
            {
                l=n; // Clamp right subarray end boundary to n
            }
            while (j < mid && k < l) // Merge while elements remain in both left and right subarrays
            {
                if (a[j] <= a[k]) // Check if element in left subarray is smaller or equal
                {
                    b[z] = a[j]; // Place smaller element from left subarray into buffer array b
                    j++; // Advance left subarray index
                }
                else
                {
                    b[z] = a[k]; // Place smaller element from right subarray into buffer array
b
                    k++; // Advance right subarray index
                }
                z++; // Increment destination buffer index
            }
            while (j < mid) // Copy any remaining unmerged elements from left subarray
            {
                b[z] = a[j]; // Append leftover left element into buffer array b
                j++; // Advance left index
                z++; // Advance buffer index
            }
            while (k < l) // Copy any remaining unmerged elements from right subarray
            {
                b[z] = a[k]; // Append leftover right element into buffer array b
                k++; // Advance right index
                z++; // Advance buffer index
            }
        }
    }
}
```

```

    for (int i = 0; i < n; i++)          // Copy fully merged pass results back from b to original array
    {
        a[i] = b[i];                    // Transfer element from buffer b[i] to array a[i]
    }
    s = s * 2;                          // Double the subarray window size for the next merge pass
}
for (int i = 0; i < n; i++)            // Loop to print final sorted output array
{
    printf("%d ", b[i]);                // Print each sorted number separated by space
}
return 0;                              // Program finished successfully
}

```